

Student 20 **Moniga V** 192521184RC

20 / 20 submitted

Q1. Void Function (No Return Value)

CODE

```
def welcome():  
    print("Welcome to Python Programming")  
  
welcome()
```

OUTPUT

(no output)

Q2. Void Function (No Return Value)

CODE

```
def table(n):  
    for i in range(1, 11):  
        print(n, "x", i, "=", n * i)  
  
num = int(input("Enter a number: "))  
table(num)
```

INPUT

5

OUTPUT

(no output)

Q3. Void Function (No Return Value)

CODE

```
def even_numbers(n):  
    for i in range(2, n + 1, 2):  
        print(i)  
  
num = int(input("Enter N: "))  
even_numbers(num)
```

INPUT

10

OUTPUT

(no output)

Q4. Fruitful Function (Returns One Value)

CODE

```
def count_vowels(s):  
    count = 0  
    for ch in s:  
        if ch.lower() in "aeiou":  
            count += 1  
    return count  
  
text = input("Enter a string: ")  
print("Number of vowels:", count_vowels(text))
```

INPUT

Hello World

OUTPUT

(no output)

SIMATS Engineering • CSE • CSA0802 — Python Programming • 29-07-2026 • Category: Class Work (28.7.26) • Student Submissions Report
Q5. Fruitful Function (Returns One Value)**CODE**

```
def largest(lst):  
    return max(lst)  
  
numbers = list(map(int, input("Enter the numbers: ").split()))  
print("Largest element:", largest(numbers))
```

INPUT

01 3023 -23 409

OUTPUT

(no output)

Q6. Fruitful Function (Returns One Value)**CODE**

```
def is_prime(n):  
  
    if n ≤ 1:  
        return False  
  
    for i in range(2, int(n**0.5) + 1):  
        if n % i == 0:  
            return False  
  
    return True  
  
# Example usage:  
number = 11  
result = is_prime(number)  
print(f"Is {number} prime? {result}")
```

OUTPUT

(no output)

Q7. Function Returning Two or More Values**CODE**

```
def calculate_student_performance(mark1, mark2, mark3):  
    total = mark1 + mark2 + mark3  
    average = total / 3  
  
    if average ≥ 90:  
        grade = "A"  
    elif average ≥ 80:  
        grade = "B"  
    elif average ≥ 70:  
        grade = "C"  
    elif average ≥ 60:  
        grade = "D"  
    else:  
        grade = "F"  
  
    return total, average, grade  
  
m1, m2, m3 = 85, 90, 78  
total_marks, avg_marks, final_grade = calculate_student_performance(m1, m2, m3)  
  
print(f"Total Marks: {total_marks}")  
print(f"Average Marks: {avg_marks:.2f}")  
print(f"Grade: {final_grade}")
```

OUTPUT

(no output)

Q8. Function Returning Two or More Values**CODE**

```
import math

def calculate_circle_properties(radius):
    area = math.pi * (radius**2)
    circumference = 2 * math.pi * radius

    return area, circumference

r = 5
circle_area, circle_circumference = calculate_circle_properties(r)

print(f"Radius: {r}")
print(f"Area: {circle_area:.2f}")
print(f"Circumference: {circle_circumference:.2f}")
```

OUTPUT

(no output)

Q9. Function Returning Two or More Values**CODE**

```
def calculate_math_properties(number):

    square = number ** 2
    cube = number ** 3
    square_root = number ** 0.5

    return square, cube, square_root

num = 16
sq, cb, sqrt = calculate_math_properties(num)

print(f"Number: {num}")
print(f"Square: {sq}")
print(f"Cube: {cb}")
print(f"Square Root: {sqrt}")
```

OUTPUT

(no output)

Q10. Keyword Arguments**CODE**

```
def student(name, age, department):
    print(f"Name: {name}")
    print(f"Age: {age}")
    print(f"Department: {department}")

student(name="Alice", age=20, department="Computer Science")
```

OUTPUT

(no output)

Q11. Keyword Arguments**CODE**

```
def employee(empid, name, salary):
    print(f"Employee ID: {empid}")
    print(f"Name: {name}")
    print(f"Salary: {salary}")

employee(salary=75000.0, empid=101, name="John Doe")
```

OUTPUT

(no output)

Q12. Keyword Arguments**CODE**

```
def book(title, author, price):
    print(f"Book Title: {title}")
    print(f"Author: {author}")
    print(f"Price: ${price:.2f}")

book(title="The Alchemist", author="Paulo Coelho", price=14.99)
```

OUTPUT

(no output)

Q13. Variable-Length Arguments (*args)**CODE**

```
def calculate_sum(*args):
    total = 0
    for num in args:
        total += num

    return total

print(calculate_sum(1, 2, 3))
print(calculate_sum(10, 20, 30, 40, 50))
print(calculate_sum(5))
```

OUTPUT

(no output)

Q14. Variable-Length Arguments (*args)**CODE**

```
def calculate_average(*args):
    if not args:
        return 0.0

    total = sum(args)
    count = len(args)
    average = total / count

    return average

print(calculate_average(85, 90, 78))
print(calculate_average(95, 88, 92, 79, 91))
print(calculate_average(80))
```

OUTPUT

(no output)

Q15. Variable-Length Arguments (*args)

CODE

```
def print_strings(*args):  
    for s in args:  
        print(s)  
  
print_strings("Apple", "Banana", "Cherry", "Date")
```

OUTPUT

(no output)

Q16. Variable-Length Arguments (*args)

CODE

```
def find_largest(*args):  
    if not args:  
        return None  
    largest = args[0]  
    for num in args:  
        if num > largest:  
            largest = num  
  
    return largest  
  
print(find_largest(10, 45, 23, 89, 12))  
print(find_largest(5, 2, 9, -3))  
print(find_largest(42))
```

OUTPUT

(no output)

Q17. Recursive Functions

CODE

```
def fibonacci(n):  
    if n ≤ 1:  
        return n  
    else:  
        return fibonacci(n - 1) + fibonacci(n - 2)  
  
def generate_fibonacci_series(n):  
    if n ≤ 0:  
        print("Please enter a positive integer.")  
    else:  
        print(f"Fibonacci series up to {n} terms:")  
        for i in range(n):  
            print(fibonacci(i), end=" ")  
  
generate_fibonacci_series(7)
```

OUTPUT

(no output)

Q18. Recursive Functions**CODE**

```
def sum_of_digits(n):
    if n == 0:
        return 0
    else:
        return (n % 10) + sum_of_digits(n // 10)

number = 12345
result = sum_of_digits(number)

print(f"The sum of the digits of {number} is: {result}")
```

OUTPUT

(no output)

Q19. Recursive Functions**CODE**

```
def reverse_string(s):
    if len(s) ≤ 1:
        return s
    else:
        return s[-1] + reverse_string(s[:-1])

text = "hello"
result = reverse_string(text)

print(f"Original String: {text}")
print(f"Reversed String: {result}")
```

OUTPUT

(no output)

Q20. Recursive Functions**CODE**

```
def power(base, exp):
    if exp == 0:
        return 1
    else:
        return base * power(base, exp - 1)

x = 2
y = 3
result = power(x, y)

print(f"{x} to the power of {y} is: {result}")
```

OUTPUT

(no output)